Vision 기본 모델 Day 3

경량 CNN · 전이학습 · Grad-CAM

HYUNDAI AI Insight Campus · 온디바이스 AI · Vision 기본 모델 3/5

2026. 10. 29 (목) 09:00–18:00 · 8교시 · Google Colab T4 GPU

지연시간 | MAC | Depthwise Separable | MobileNetV3 | 전이학습 | Grad-CAM

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 정확도 vs 속도

CPU · 스레드 1 · 배치 1 · warm-up 조건으로 지연시간을 재고, 정확도 · 파라미터 · MAC 과 함께 비교한다

2 연산량 · DS Conv

1×1 bottleneck (약 1/17) 과 Depthwise Separable (약 1/8~1/9) 이 연산을 줄이는 원리를 공식과 코드로 확인한다

3 MobileNet V1 · V2 · V3

DS Conv → inverted residual · linear bottleneck → SE · h-swish 로 이어지는 경량화 설계를 직접 구현한다

4 전이학습

ImageNet 사전학습 MobileNetV3 를 feature extraction · fine-tuning 하고, 적은 데이터에서의 효과를 실험으로 본다

5 Grad-CAM

forward hook 과 기울기 hook 으로 Grad-CAM 을 구현해 모델이 이미지의 어디를 보고 판단했는지 확인한다

6 미니 프로젝트

MobileNetV3 전이학습 (5 epoch, test ≥ 0.85) 과 Day 2 ResNet-20 을 정확도 · MAC · 지연시간으로 비교한다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 연산량 → DS Conv, 오후 MobileNet → 전이학습 → Grad-CAM → 미니 프로젝트

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 2 복습 퀴즈 · 정확도 vs 속도	퀴즈 + 강의	D3-1 정확도 vs 속도
2	10:00–10:50	연산량 계산 — 파라미터 · MAC · 메모리 · 1×1	강의 + 실습	D3-2 연산량 지도
3	11:00–11:50	Depthwise Separable Convolution	강의 + 실습	D3-3 Depthwise + Pointwise
4	13:00–13:50	MobileNet V1 · V2 · V3	강의 + 실습	D3-4 Inverted residual · SE
5	14:00–14:50	전이학습 — feature extraction vs fine-tuning	강의 + 실습	D3-5 전이학습 — 동결과 미세조정
6	15:00–15:50	적은 데이터와 전이학습	강의 + 실습	D3-6 적은 데이터 실험
7	16:00–16:50	Grad-CAM — 모델은 어디를 보는가	강의 + 실습	D3-7 Grad-CAM 계산
8	17:00–17:50	미니 프로젝트 — MobileNetV3 전이학습 vs ResNet-20	개인 실습	D3-8 모델 고르기 — 비교표

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 검증 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 3)

https://imguru-mooc.github.io/AI_CAMPUS/VB-day3/

animation.html	애니메이션 8 개 (D3-1 ~ D3-8) — #d3-7 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기
VB_Day3_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab · T4 GPU)
VB_Day3.pdf	오늘 슬라이드 PDF
resnet20_cifar10.pt	Day 2 강사 ResNet-20 가중치 (VB-day2/data) — 노트북이 1 교시에 내려받음
Day 2 복습 퀴즈	09:00 시작 — 강사가 Google Forms 링크 공유 (20 문항 · 15 분)

노트북은 " 파일 → Drive 에 사본 저장 " 후 작업 · 런타임 → 런타임 유형 변경 → T4 GPU · 하드웨어 없이 데이터만으로 진행합니다

오늘의 큰 그림 — ResNet-50 에서 MobileNetV3 까지

같은 ImageNet 분류인데 연산량은 72 배 차이 — 정확도와 MAC(속도) 을 함께 보세요 (224×224 입력)

ResNet-50 2015

params 25,557,032

top-1 76.1%

MAC 4,089,184,256

bottleneck 블록 · 102.2 MB

→

ResNet-18 2015

params 11,689,512

top-1 69.8%

MAC 1,814,073,344

basic 블록 · Day 2 의 큰 형

→

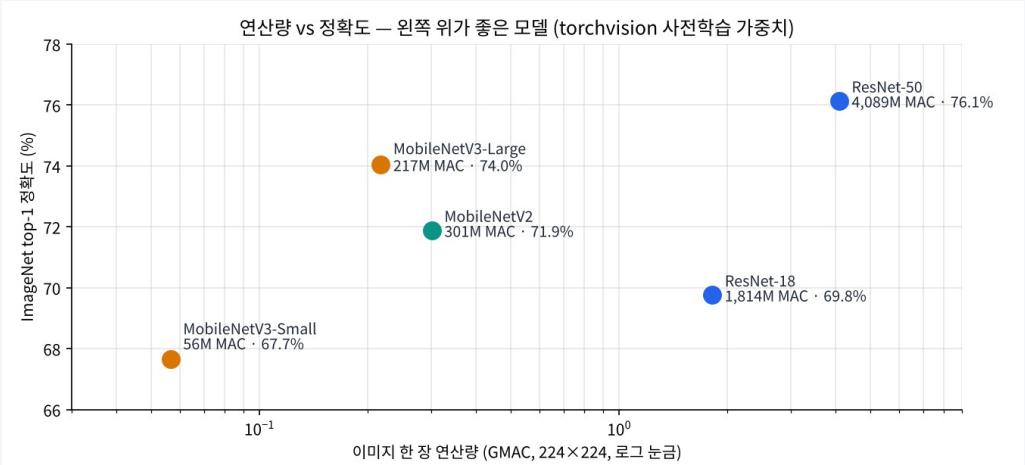
MobileNetV3-Small 2019

params 2,542,856

top-1 67.7%

MAC 56,510,400

DS Conv · SE · h-swish (1/72)



오늘의 데이터 — CIFAR-10

- 원래 계획한 ' 현장 안전 이미지셋 ' 대신 Day 2 와 같은 CIFAR-10
- 처음부터 학습한 ResNet-20 과 공정하게 비교
- MobileNet 입력은 Resize 96 + ImageNet 정규화
- 코드는 ImageFolder 로 그대로 교체 가능 (6 교시)

1 Day 2 복습 퀴즈 · 정확도 vs 속도

09:00–09:50

학습 목표

- 09:00–09:15 Vision Day 2 복습 퀴즈 (20 문항)
- 온디바이스 = 정확도 + 속도 + 메모리
- 측정 규칙 : CPU · 스레드 1 · 배치 1 · warm-up 3 회 버림 · 여러 번 평균
- 30 FPS = 한 장 33 ms 이내

노트북 실습

1. 문제 1-1 Day 2 ResNet-20 가중치 test
2. 문제 1-2 latency_ms 함수
3. 문제 1-3 네 모델 비교표

▶ 애니메이션 D3-1 정확도 vs 속도

핵심 개념

1-1

```
resnet20_cifar10.pt (Day 2, 20 epoch) 내려받아 load_state_dict
test 0.8587 (실행 예 · Day 2 사전 실행 0.8587)
```

1-2 latency_ms

```
deepcopy → cpu() → eval() · torch.set_num_threads(1)
no_grad · warm-up 3 회 버림 · perf_counter 로 reps=30 평균
스레드 수 원복 → ms 반환
```

1-3 비교

	params	MAC	CPU ms (실행 예)
ResNet-20 (32)	272,474	40,813,184	2.87
ResNet-18 (224)	11,689,512	1,814,073,344	70.4
ResNet-50 (224)	25,557,032	4,089,184,256	134
MobileNetV3-S (224)	2,542,856	56,510,400	8.75

ImageNet top-1

```
ResNet-18 69.8% · ResNet-50 76.1% · MobileNetV3-S 67.7%
```

체크포인트

ResNet-50 MAC = MobileNetV3-Small 의 약 72 배, 정확도 차이는 8.4%p. 지연시간은 장치 · 라이브러리 · 스레드 수에 따라 달라지므로 같은 규칙으로 재서 비교한다.

2 연산량 계산 — 파라미터 · MAC · 메모리 · 1×1

10:00–10:50

학습 목표

- 파라미터 메모리 = params × 4 B (float32)
- 활성화 메모리 = Conv · Linear 출력 원소 합 × 4 B
- MAC = 곱 - 누산 횟수 — 속도의 대략적 지표
- 1×1 conv = 위치마다 채널만 섞는 압축기

노트북 실습

1. 문제 2-1 model_cost 함수
2. 문제 2-2 basic vs bottleneck 블록

▶ 애니메이션 D3-2 연산량 지도

핵심 개념

2-1 model_cost

```
forward hook 으로 Conv · Linear 출력 out.numel() 합산
→ params · macs · param_MB · act_MB
ResNet-50 (224): param_MB = 25,557,032 × 4 B = 102.2 MB
```

2-2



2-2 계산 (256 채널)

```
basic       3×3 256→256 ×2 = 2·9·256² = 1,179,648
bottleneck  1×1 16,384 + 3×3 36,864 + 1×1 16,384 = 69,632
→ 1,179,648 / 69,632 = 16.9 ≈ 17 배  (bias 없음 · BN 제외)
```

체크포인트

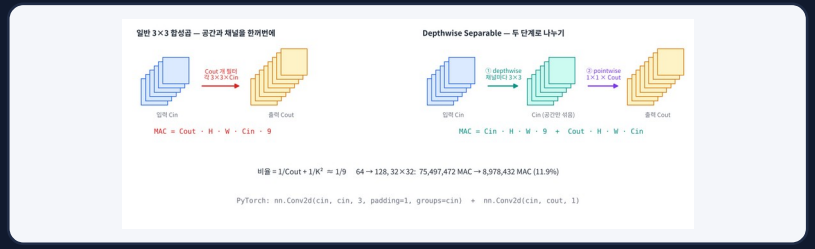
비싼 3×3 을 채널이 적은 곳 (64) 에서 하고 1×1 로 줄였다 늘린다 . MobileNet 도 1×1 conv 가 핵심 부품 — 3 교시 pointwise 가 바로 1×1 이다 .

3 Depthwise Separable Convolution

11:00–11:50

학습 목표

- depthwise 3×3 (groups = Cin) — 채널마다 공간만
- pointwise 1×1 — 채널만 섞기
- 연산 비율 = 1/Cout + 1/K² (K=3 → 약 1/9~1/8)
- MAC 이 줄어도 실제 속도는 장치에서 잰다



노트북 실습

1. 문제 3-1 DSConv 블록
2. 문제 3-2 가중치 · MAC 비교
3. 문제 3-3 CPU 지연시간

▶ 애니메이션 D3-3 Depthwise + Pointwise

핵심 개념

MAC

```
일반  Cout · H · W · Cin · 9
DS    Cin · H · W · 9 + Cout · H · W · Cin  → 비율 1/Cout + 1/9
```

3-1 DSConv

```
dw: Conv2d(cin, cin, 3, stride, padding=1, groups=cin, bias=False)
    + BN + ReLU
pw: Conv2d(cin, cout, 1, bias=False) + BN + ReLU
```

3-2 64→128 32×32

```
가중치  일반 73,728   DS 8,768 (dw 576 + pw 8,192) = 11.9%
MAC     일반 75,497,472   DS 8,978,432 (dw 589,824 + pw 8,388,608)
= 0.1189 = 1/128 + 1/9 · pointwise 가 DS 연산의 93%
```

3-3 (실행 예)

```
CPU 1 스레드 · 입력 64×56×56 · MAC 은 8.4 배 차이
일반 5.14 ms  vs  DS 1.26 ms → 4.1 배
```

체크포인트

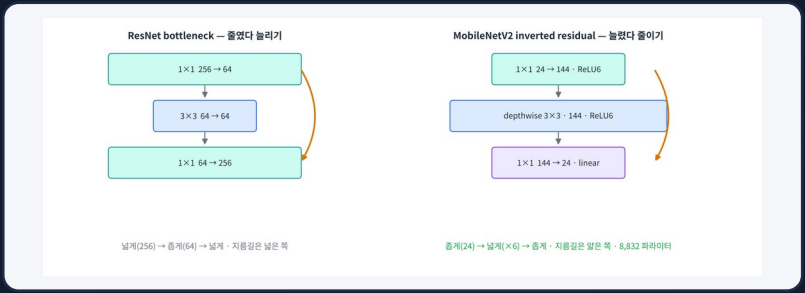
depthwise 는 계산 대비 메모리 이동이 많다 (메모리 병목) → MAC 8.4 배 감소가 그대로 속도 8.4 배가 되지 않는다. 지연시간은 실제 장치에서 재야 한다.

4 MobileNet V1 · V2 · V3

13:00–13:50

학습 목표

- V1 (2017) — DS Conv 쌓기
- V2 (2018) — inverted residual · linear bottleneck
- V3 (2019) — NAS + SE + h-swish



노트북 실습

1. 문제 4-1 InvertedResidual 블록
2. 문제 4-2 h-swish 구현 · 비교
3. 문제 4-3 mobilenet_v3_small 구조

▶ 애니메이션 D3-4 Inverted residual · SE

핵심 개념

4-1 Inverted Residual

```
1×1 24→144 (×t=6) · ReLU6 → dw 3×3 · ReLU6 → 1×1 144→24 ( 활성화 없음 )
(24,24,s1,t6) 8,832 = 3,456 + 288 + 1,296 + 288 + 3,456 + 48
(16,24,s2) 5,136 — stride 1 & cin == cout 일 때만 지름길
```

4-2 h-swish

```
h-swish(x) = x · ReLU6(x + 3) / 6   (x ≤ −3 → 0, x ≥ 3 → x)
swish(x) = x · σ(x) 의 근사 · nn.Hardswish
```

SE

```
채널별 평균 (squeeze) → FC 두 개 → hard-sigmoid 0~1
→ 채널마다 곱 (excitation) — 중요한 채널을 키운다
```

4-3 V3-Small

```
features 안 InvertedResidual 11개 중 SE 9개 · 마지막 (576, 7, 7)
classifier: Linear(576→1024) · Hardswish · Dropout · Linear(1024→1000)
전체 2,542,856 · 224 입력 56,510,400 MAC (ResNet-50 의 1/72)
```

체크포인트

ResNet bottleneck = 줄였다 늘리기 (256→64→256, 지름길이 넓은 쪽), V2 = 늘렸다 줄이기 (24→144→24, 지름길이 얇은 쪽) → inverted. 얇은 쪽엔 ReLU 를 빼서 정보 손실을 막는다 .

5 전이학습 — feature extraction vs fine-tuning

14:00–14:50

학습 목표

- ImageNet 1,000 클래스 · 128 만 장으로 배운 features 재사용
- classifier[3] 만 10 클래스로 교체
- feature extraction = features 동결 (requires_grad=False)
- fine-tuning = 전체를 작은 학습률로 함께 학습

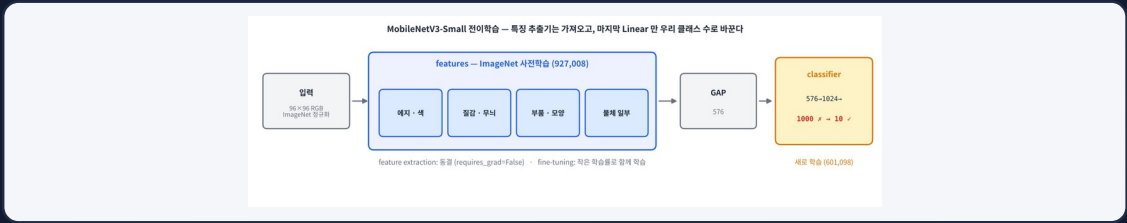
노트북 실습

1. 문제 5-1 사전학습 모델 · 분류기 교체
2. 문제 5-2 feature extraction 1 epoch
3. 문제 5-3 fine-tuning vs 처음부터

▶ 애니메이션 D3-5 전이학습 — 동결과 미세조정

핵심 개념

구조



5-1

```
W = MobileNet_V3_Small_Weights.IMAGENET1K_V1
mnet = mobilenet_v3_small(weights=W)
mnet.classifier[3] = nn.Linear(1024, 10)
전체 1,528,106 = features 927,008 + classifier 601,098
96 입력 MAC 11,057,600
```

5-2 · 5-3

증강 없는 train 50,000 장 · Adam 1e-3

방법 (1 epoch)	학습 파라미터	test (실행 예)
feature extraction	601,098 (39%)	0.6607
fine-tuning	1,528,106	0.8273
처음부터 (weights=None)	1,528,106	0.5470

체크포인트

주의 : ImageNet 정규화 (mean 0.485 · 0.456 · 0.406) · 입력이 너무 작으면 안 됨 (32 → Resize 96) · 동결해도 BN 의 running 통계는 train() 모드에서 바뀐다 .

6 적은 데이터와 전이학습

15:00–15:50

학습 목표

- 클래스당 100 장 = 1,000 장 균형 샘플 (전체의 2%)
- 같은 조건 : 증강 · Adam 1e-3 · 배치 64 · 10 epoch
- 데이터가 적을수록 사전학습 효과가 크다
- 현장 데이터는 ImageFolder 로 그대로

노트북 실습

1. 문제 6-1 균형 1,000 장 인덱스
2. 문제 6-2 처음부터 vs 사전학습
3. 문제 6-3 클래스별 정확도

▶ 애니메이션 D3-6 적은 데이터 실험

핵심 개념

6-1

```
t = np.array(train_mn.targets)
idx = np.concatenate([np.where(t == c)[0][:100]
                      for c in range(10)])
np.bincount(t[idx]) → 클래스마다 100 · 합 1,000
```

6-2 (실행 예)

```
1,000 장 · 10 epoch → test 10,000 장
처음부터 0.3560   vs   사전학습 0.6561
확인 셀 : 차이 > 0.15
```

6-3

```
클래스별 정확도 = 사전학습 − 처음부터
가장 크게 오른 클래스 ( 실행 예 ): frog (+0.59), bird (+0.43), dog (+0.38)
```

ImageFolder

```
safety/train/helmet/*.jpg · safety/train/no_helmet/*.jpg
ds = ImageFolder('safety/train', transform=tf)
ds.classes = 폴더 이름 순서 = 레이블 번호
```

체크포인트

처음부터 학습하면 1,000 장으로 에지 · 질감부터 배워야 한다 . 사전학습 features 는 이미 에지 · 질감 · 부품을 안다 → 현장 데이터가 적을 때 첫 선택은 사전학습 + fine-tuning.

7 Grad-CAM — 모델은 어디를 보는가

16:00–16:50

학습 목표

- 마지막 conv 출력 A^k 와 기울기 ∂ y^c/∂A^k
- α_k = 기울기의 공간 평균 (GAP)
- CAM = ReLU(Σ α_k A^k) → 0~1 → 업샘플 → 겹쳐 그리기
- 오답 : 예측 클래스 CAM vs 정답 클래스 CAM

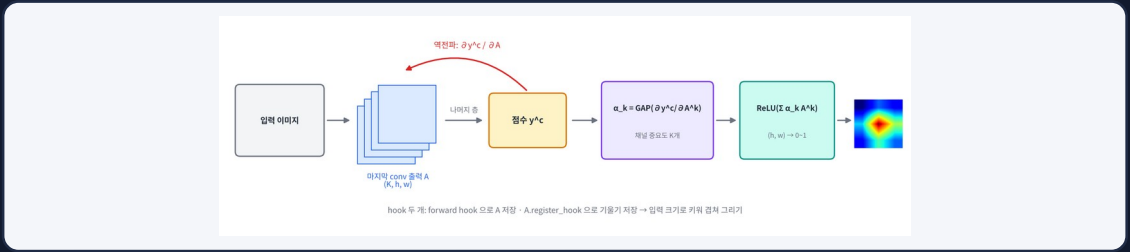
노트북 실습

1. 문제 7-1 GradCAM 클래스 (hook)
2. 문제 7-2 오답에서 CAM 비교
3. 문제 7-3 MobileNet CAM 크기

▶ 애니메이션 D3-7 Grad-CAM 계산

핵심 개념

계산 흐름



7-1

```
forward hook: A 저장 + out.register_hook( 기울기 저장 )
logits[torch.arange(N), cls].sum().backward()
alpha = grads.mean(dim=(2, 3), keepdim=True)
cam = F.relu((alpha * acts).sum(dim=1))   # → 정규화 → 업샘플
```

CAM 크기

```
ResNet-20 layer3          (32 입력 , stride 4)   → 8×8
MobileNetV3-S features[-1] (96 입력 , stride 32) → 3×3
```

체크포인트

ReLU 는 그 클래스 점수를 올리는 위치만 남기기 위해 . 다 쓴 hook 은 remove(). 오답에서 배경 · 맥락에 속았는지 본다 . 3×3 CAM 은 거칠다 → 위치가 중요하면 탐지 (Day 4).

8 미니 프로젝트 — MobileNetV3 전이학습 vs ResNet-20

17:00–17:50

강의 노트북 8 교시 · MobileNetV3-S + AdamW + OneCycleLR

```
EPOCHS = 5
mn_loader = DataLoader(train_mn, batch_size=128,
                       shuffle=True)        # 391 step
model = mobilenet_v3_small(weights=W_MNV3S)
model.classifier[3] = nn.Linear(1024, 10)
opt = AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
sched = OneCycleLR(opt, max_lr=1e-3, epochs=EPOCHS,
                   steps_per_epoch=len(mn_loader))
```

과제	내용	배점
1	증강 로더 · 사전학습 모델 · AdamW + OneCycleLR	20
2	fine-tuning 5 epoch — test ≥ 0.85 (GPU)	30
3	비교표 — params · MAC · CPU 지연시간 · test	20
4	오답 8 장 Grad-CAM (3×3)	10
5	결과 해석 (마크다운)	10 + 10

채점 셀 실행 → 자동 채점 80 점 (과제 1~4) + 강사 20 점 (과제 5)

train 50,000 장 · Resize 96 · 증강 · 391 step/epoch

▶ D3-8 모델 고르기 — 비교표 — 먼저 보고 코드로 확인

목표 test ≥ 0.85 (5 epoch, T4 약 5 분 예상)

강사 사전 실행 5 epoch test 0.8842 (실행 예) · T4 약 1 분 /epoch

모델 고르기 — CIFAR-10, 이미지 한 장 기준

	ResNet-20	MobileNetV3-S
학습 · 입력	처음부터 · 32	전이학습 · 96
params	272,474	1,528,106
MAC	40,813,184	11,057,600
CPU ms (실행 예)	2.87	6.18
test (실행 예)	0.8587	0.8842

크기 파라미터는 MobileNet 이 5.6 배 , MAC 은 약 1/3.7

해상도 MAC 은 입력 크기 제곱에 비례 — 96→224 ≈ 5.4 배

속도 지연시간은 실제 장치에서 측정 (depthwise 메모리 병목)

현장 데이터가 적으면 사전학습 경량 모델 + fine-tuning + Grad-CAM 확인

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

정확도 vs 속도	CPU · 스레드 1 · 배치 1 · warm-up · ResNet-50 MAC 은 MobileNetV3-S 의 72 배	D3-1
연산량	메모리 = params × 4 B · 1×1 bottleneck 69,632 vs basic 1,179,648 (약 1/17)	D3-2
DS Conv	depthwise (groups=Cin) + pointwise 1×1 · 비율 1/Cout + 1/K² · 속도는 장치에서 측정	D3-3
MobileNet	V1 DS Conv · V2 inverted residual + linear bottleneck · V3 SE + h-swish	D3-4
전이학습	classifier[3] → 10 · feature extraction (601,098 학습) vs fine-tuning (전체)	D3-5
적은 데이터	클래스당 100 장 · 10 epoch — 사전학습이 처음부터보다 크게 높다 · ImageFolder	D3-6
Grad-CAM	α_k = GAP(∂y^c/∂A^k) · CAM = ReLU(Σ α_k A^k) · ResNet-20 8×8 / MobileNet 3×3	D3-7
미니 프로젝트	MobileNetV3-S 전이학습 5 epoch — test ≥ 0.85 · params · MAC · 지연시간 비교표	D3-8

Day 4 로 가는 다리 — 객체 탐지

분류기는 '무엇이' 만 답합니다. 다음은 '무엇이 어디에' — bbox · IoU · NMS · mAP · YOLO

Day 3 (오늘)		Day 4 (11/2 월)
분류 : 이미지 → 클래스 1 개	→	탐지 : 물체마다 클래스 + bbox
Grad-CAM 3×3 — 위치가 거칠다	→	위치는 탐지로 — bbox 를 직접 예측
accuracy (맞음 / 틀림)	→	IoU · mAP — 얼마나 겹치는가
softmax 로 클래스 하나	→	NMS — 겹치는 박스 정리
MobileNetV3 전이학습	→	YOLO — 사전학습 탐지기로 사람 찾고 세기
latency_ms · model_cost	→	**그대로 사용**

bbox → IoU → NMS → mAP → YOLO

이후 : Day 5 (11/3 화) 세그 · 포즈 · ONNX → ONNX · TensorRT (11/4~) → Quantization · Pruning (11/16~)